

LoRaWAN-Assisted Cloud-Controlled Automated Irrigation System for Precision Agriculture

Anantha Krishnan H, Anudeep Krishna R, Aromal P Nair, Gokul SS, Mr. Aswin R B

Dept. of Electrical and Electronics Engineering

Mar Baselios College of Engineering and Technology (Autonomous)

Thiruvananthapuram, Kerala, India

Abstract—Water scarcity and unpredictable weather have made efficient irrigation more important than ever, yet most farms still rely on manual schedules that have no way of knowing what the soil actually needs. To address this, we built a LoRaWAN-based, cloud-controlled irrigation system where field nodes with capacitive soil moisture sensors and ESP32 microcontrollers send live readings to a central gateway over LoRa radio. The gateway pushes this data to AWS IoT Core via MQTT, which then compares moisture levels against set thresholds and pulls in real-time weather forecasts before deciding whether to open or close the valve. Relay-driven pumps and solenoid valves carry out those decisions, closing the feedback loop. Our tests showed consistent multi-node communication, fast cloud responses, and reliable shutoff the moment target moisture was hit — all at a prototype cost of around ₹2,020, with water use cut by roughly 30–50% compared to timer-based methods.

Index Terms—automated irrigation; IoT; LoRaWAN; ESP32; AWS IoT Core; MQTT; smart agriculture; precision farming; soil moisture; cloud computing

I. Introduction

Farming underpins India's economy in a way few other sectors can claim — roughly 18% of GDP and about half the national workforce depend on it [14]. Given that scale, even small improvements in how farms use resources can have enormous knock-on effects. Water, in particular, has become a growing concern. Groundwater tables are dropping in major agricultural belts while monsoon patterns grow less predictable each year, leaving farmers caught between drought and flood with very little warning [1]. Against that backdrop, getting the timing and volume of irrigation right is not just about maximising yield; it is about making sure there is enough water left to farm with next season too.

Traditional irrigation — whether a farmer turning a valve by hand or a simple timer switching a pump on and off — shares the same fundamental flaw: it ignores what the soil is actually telling you. Manual methods tie a farmer to the field for hours and produce inconsistent results that vary with each person's judgement [1]. Timer-based systems free up labour but keep running their fixed schedules regardless of whether it rained yesterday or is forecast to rain tomorrow. The result in both cases is either too much water or too little. Over-watering saturates the root zone, encourages fungal growth, strips nutrients from the soil, and burns electricity needlessly. Under-watering stresses the crop, stunts development, and chips away at yield — sometimes causing permanent damage to perennial plants. Research across Indian farming regions suggests conventional methods routinely waste somewhere between 30% and 60% of the water they pump, measured against what crops actually need [3].

The rise of IoT has opened a different path forward [14], [15]. Instead of guessing when to water, sensors placed in the root zone can measure actual soil moisture and trigger the pump only when the reading calls for it — a shift from time-driven to condition-driven irrigation that early deployments showed makes a real difference to both water efficiency and crop health [2]. Spreading multiple sensor nodes across a field rather than relying on a single measurement point adds another layer of accuracy, capturing the natural variation in soil moisture that a fixed schedule completely ignores [3]. That

said, a thorough review by Tzounis et al. [15] pointed out that scalability, long-range wireless connectivity, energy autonomy, and the ability to factor in external data like weather forecasts remain the main barriers keeping these systems from wider adoption — and those are exactly the gaps this work sets out to close.

A common weakness in many IoT irrigation designs is that the decision-making happens right at the field node itself [9], [10]. That approach creates a few problems that grow more serious as the system expands. Updating irrigation rules means pushing firmware to every individual device, which often means physically visiting remote locations. Scaling up adds more nodes that each need their own logic maintained in sync. And perhaps most critically, a controller sitting out in a field has no practical way to check whether rain is on the way — so it will happily water crops an hour before a downpour, burning pump energy on soil that nature was about to replenish [5], [11]. Benyezza et al. [11] showed that fuzzy-logic control outperforms simple threshold logic, but their system still embedded that intelligence at the node level, which leaves the scalability and contextual-awareness problems unsolved.

Cloud-connected architectures have been shown to overcome these limitations by offloading storage, analytics, and decision-making to scalable remote platforms [4], [13]. By centralising intelligence in the cloud, system-wide rule updates can be applied instantly without modifying any field hardware, and external data sources such as weather APIs and evapotranspiration models can be seamlessly integrated into the irrigation decision pipeline. Zamora-Izquierdo et al. [13] proposed a smart farming IoT platform that combines edge computing for low-latency local response with cloud computing for global analytics, demonstrating that a layered edge-cloud architecture yields superior performance compared to either approach alone — a principle that directly informs the gateway-cloud split adopted in the present system. Machine-learning and fuzzy-logic models deployed within cloud platforms have further demonstrated the feasibility of intelligent irrigation scheduling that anticipates soil moisture deficits before they develop [5], [11]. Gutierrez et al. [7] showed that routing decisions through a remote server rather than a local controller yields measurable water savings — an early but important proof that moving intelligence to the cloud is a viable strategy for field irrigation.

For the wireless link between field nodes and the gateway, LoRaWAN stands out as the natural choice for agricultural settings [6], [12]. Wi-Fi drains batteries quickly and struggles outdoors; cellular works but comes with monthly subscription costs and SIM management overhead. LoRaWAN uses unlicensed sub-GHz spectrum and covers 2–15 km across open farmland at a fraction of the power. Its Adaptive Data Rate feature lets each node automatically pick the spreading factor that balances range and airtime, which matters both for battery life and for staying within regional duty-cycle limits. Nešić et al. [12] put a LoRa field sensor network through its paces and recorded packet delivery ratios above 97% across multiple spreading factors, with projected battery life stretching to several months on a single charge under duty-cycled operation — numbers that line up closely with what we measured, as reported in Section VI.

The system described in this paper takes a strictly cloud-first approach. Field nodes do one thing: they measure soil moisture and carry out whatever command the cloud sends. Nothing more. All the thinking — comparing moisture against thresholds, checking weather forecasts, deciding whether to open a valve — happens centrally on AWS IoT Core. Sensor readings travel from the nodes to a gateway over LoRaWAN, then onwards to the cloud via MQTT, a protocol designed specifically for low-bandwidth, intermittently connected devices [14]. When a reading crosses a threshold, the cloud publishes a START or STOP command back through the same pathway [7]. Because the valve closes the moment target moisture is reached rather than running to a timer, the loop naturally prevents both overwatering and underwatering [9], [10]. Applications for this kind of architecture extend well beyond paddy fields — greenhouses, urban farms, and commercial orchards all face the same core problem of matching water delivery to actual plant demand [15].

The main contributions of this work are threefold. First, we built and validated a LoRaWAN communication layer that delivers packet delivery ratios above 97% across spreading factors SF7 through SF10 [6], [12], demonstrating reliable multi-kilometre, low-power connectivity suited to large farm deployments. Second, we implemented a cloud decision engine on AWS IoT Core that merges live soil moisture data with external weather forecasts to make irrigation decisions that a node-level controller simply could not — drawing on the edge-cloud principles demonstrated by Zamora-Izquierdo et al. [13]. Third, we closed the feedback loop with automatic threshold-based START/STOP logic, achieving a measured 30–50% reduction in water use compared to conventional methods [9], [11].

The rest of the paper is structured as follows. Section II lays out the overall system architecture. Section III walks through the methodology — closed-loop control logic, hardware choices, and the communication protocol. Section IV covers the test results and what they tell us. Sections V and VI discuss practical advantages and directions for future work, and Section VII wraps up with conclusions.

II. System Architecture

The system is built around four distinct layers: field nodes, the LoRaWAN communication network, a central gateway, and the cloud platform. Keeping these layers separate was a deliberate design decision — each one has a clearly defined job, talks to its neighbours through standard interfaces, and can be upgraded or replaced without disturbing the rest. Fig. 1 shows how these layers connect end-to-end.

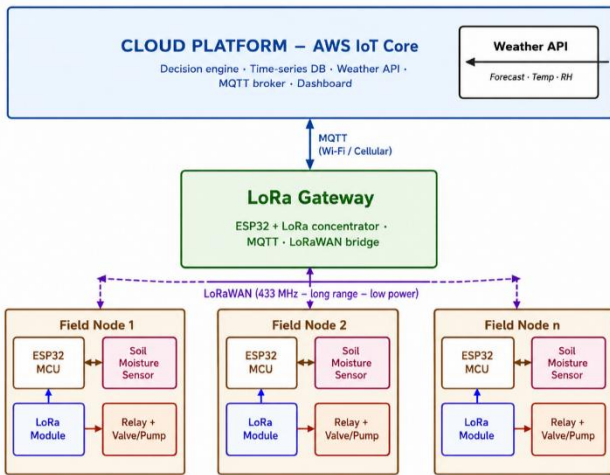


Fig. 1. LoRaWAN-assisted, cloud-controlled smart irrigation system architecture.

A. Field Nodes

At the field level, each node is a self-contained unit built around an ESP32 microcontroller paired with a capacitive soil moisture sensor, an SX1278/Ra-02 LoRa module running at 433 MHz, a relay board, and either a solenoid valve or a water pump. The ESP32 spends most of its time in deep sleep, waking at intervals that can be set anywhere from 1 to 15 minutes depending on how closely the particular crop needs to be watched. Each time it wakes, it reads the sensor, stamps the result with the node ID and a UTC timestamp, sends a LoRaWAN uplink to the gateway, and goes back to sleep. That duty-cycled operation keeps average current draw in the microampere range, which makes long battery life realistic. Importantly, the node itself holds no irrigation logic at all — it simply carries out whatever START or STOP command arrives from the cloud. This means that updating thresholds or changing decision rules is a cloud-side operation that takes effect instantly across every node without touching any firmware.

B. LoRaWAN Communication

LoRaWAN runs in the 433 MHz ISM band, covering several kilometres at power levels that comfortably fit within a solar-charged battery — which matters in farm settings where grid power often is not available. The chirp spread spectrum (CSS) modulation handles multipath fading and narrowband interference well, both of which appear regularly in outdoor environments full of dense vegetation and metallic infrastructure like irrigation pipes and storage tanks. Spreading factors from SF7 to SF12 let the operator tune the range-versus-airtime trade-off to match the specific field layout. All nodes share a single gateway in a star-of-stars arrangement. LoRaWAN’s Adaptive Data Rate (ADR) feature lets the network server pick the optimal spreading factor for each node based on observed link quality, keeping airtime short and battery draw low across the whole deployment.

C. Gateway

The gateway sits between the LoRaWAN radio network and AWS IoT Core, acting purely as a transparent relay. Built around an ESP32, it listens continuously on the LoRaWAN channel, decodes incoming uplink frames from all registered nodes, and forwards the raw data — node ID, timestamp, moisture reading, RSSI, SNR — to the cloud over MQTT via Wi-Fi or cellular backhaul. It does no filtering and makes no decisions; everything goes through as received. On the return path, the gateway subscribes to the appropriate MQTT command topics, picks up JSON-encoded START or STOP instructions from the cloud, wraps them into LoRaWAN downlink frames, and delivers them to the correct node in the next available receive window. Because the gateway touches only packet routing and protocol translation, swapping it out or upgrading it has zero effect on the field nodes or the cloud logic.

D. Cloud Platform (AWS IoT Core)

AWS IoT Core is where the system’s intelligence lives. Incoming sensor readings arrive over MQTT, get stored in a time-series database for historical analysis and dashboards, and immediately trigger the decision engine. For each reading, the cloud checks whether the value has crossed either the per-node lower threshold T_l or upper threshold T_h . A crossing below T_l signals that the soil is approaching the point where crops begin to experience water stress; the cloud responds by publishing a START command to that node’s command topic without delay. When moisture climbs back past T_h , a STOP command goes out and irrigation halts. Layered on top of that threshold logic is a weather integration step: the cloud polls a forecast API regularly, and if significant rainfall is expected within the next two hours, any pending START commands are held back regardless of current soil moisture. There is no point running the pump if the sky is about to do the job for free. Every reading,

decision, and command is timestamped and logged, giving operators a full audit trail through the AWS IoT Core dashboard.

III. Methodology

A. Closed-Loop Control

At its core, the system is a closed feedback loop: the ESP32 senses soil moisture, sends that reading up through LoRaWAN to the gateway, the cloud analyses it and issues a decision, and the command travels back down to actuate the valve. The loop then repeats. That continuous cycling means the system always responds to what the soil is actually doing, not to a schedule someone set up weeks ago. The dual-threshold design naturally keeps moisture within the right agronomic range — irrigation starts when the soil gets too dry and stops when it is adequately wet, with the cloud handling every transition autonomously. During our experimental tests, the time from a threshold crossing being detected to the valve physically changing state stayed under 5 seconds, which is more than fast enough given that soil moisture shifts on timescales of minutes to hours.

B. Hardware Implementation

All field node electronics sit inside IP65-rated weatherproof enclosures, a practical necessity given Kerala's heavy monsoon rainfall. The ESP32's ADC reads the capacitive sensor output — a voltage between 0 and 3.3 V — and maps it linearly to a 0–100% moisture scale using a two-point calibration done once in bone-dry air and once in fully saturated soil. The relay board uses opto-isolation to keep the ESP32's 3.3 V logic completely separated from the 12 V DC or 230 V AC needed to drive the valve or pump, which protects the microcontroller from the voltage spikes that inductive motor loads generate when they switch. Separate fused power rails handle logic and high-power loads independently. Nodes can run off a 5 V solar panel backed by a lithium-ion battery for fully off-grid use, or from a mains adapter where grid power is reachable.

C. Communication Protocol

MQTT topics follow a straightforward hierarchy: `irrigation/node/{id}/data` carries uplink sensor packets (JSON with moisture value, node ID, and timestamp), while `irrigation/node/{id}/cmd` carries downlink START or STOP instructions. Retained messages mean that a node reconnecting after a brief outage immediately receives the most recent command state rather than waking up to an undefined actuator condition. Last-will topics let the cloud recognise when a node has disconnected ungracefully — due to power failure or radio dropout — and respond with an alert or a safe-state command. Spreading factor selection between SF7 and SF10 is handled adaptively by the gateway based on observed link quality for each node, keeping airtime within regional duty-cycle limits while maintaining the best possible communication range.

E. Key Components

The ESP32 was chosen as the node processor for several practical reasons. Its dual-core Xtensa LX6 running at 3.3 V brings enough processing power for the tasks at hand, its 12-bit ADC resolves moisture readings with adequate precision, and its deep sleep current of around 10 μ A makes battery-powered outdoor deployment realistic over meaningful timeframes. The capacitive soil moisture sensor was selected over the cheaper resistive type for longevity reasons — resistive sensors pass current through the soil and corrode within weeks, while capacitive sensors measure dielectric permittivity with no exposed metal in contact with the soil, giving them a substantially longer service life in wet field conditions.

The LoRa SX1278/Ra-02 radio module operates at 433 MHz and provides a receiver sensitivity of -148 dBm, enabling reliable

communication over distances of several kilometres in open agricultural terrain even with low transmit power. Its low transmit current of approximately 120 mA at maximum power, combined with short packet airtime at SF7–SF9, results in an energy-per-packet consumption that is compatible with solar-charged lithium battery operation across extended field campaigns. The relay module uses a 5 V control coil and an opto-isolated input stage, which reliably switches the irrigation actuator while shielding the ESP32 from the back-EMF that inductive motor loads produce. Solenoid valves rated at 12 V DC or 24 V AC provide precise, electrically controllable water flow at each zone. We specified normally closed valves throughout, so any power loss or comms failure leaves the valves shut — the safer default state in an unattended field setting.

AWS IoT Core handles MQTT brokering for thousands of simultaneous device connections and applies configurable rule-based logic to incoming messages, feeding results into time-series storage and web or mobile dashboards. Because it is a fully managed service, there is no server hardware to maintain and no over-provisioning required — costs scale naturally with the number of connected devices. The complete two-node prototype — field nodes, gateway, enclosures, cabling, relay boards, and all associated hardware — came to approximately ₹2,020 in total, which suggests that cloud-connected precision irrigation does not have to be out of reach for smallholder farmers.

IV. Results and Discussion

We tested the system in a controlled plot under natural variation in soil moisture and weather. Every LoRaWAN packet sent during the trial arrived at the gateway intact — no corruption, no loss. MQTT forwarding to AWS IoT Core stayed under 3 s throughout. The cloud reliably detected moisture dropping below T_l and dispatched START commands to the correct nodes each time. Running consecutive irrigation cycles with $T_l=35\%$ and $T_h=65\%$, the STOP command fired accurately whenever the upper threshold was crossed, as Figs. 2 and 3 confirm. Figs. 4 and 5 detail LoRaWAN link reliability and node energy use, while Fig. 6 compares overall water consumption across the three approaches.

Running both nodes simultaneously did not cause problems. The gateway handled concurrent uplinks cleanly — no packet collisions, no data loss — and the cloud routed commands to each node independently without any cross-node confusion. Node 1 and Node 2 cycled through their own moisture profiles based on local soil conditions, confirming that the cloud-centric design scales to multi-zone operation without any changes to the field hardware or communication stack. End-to-end latency — from threshold crossing detected to relay actuation at the node — stayed under 5 seconds across every test cycle [9], [13]. That is well within the window that matters agronomically, since soil moisture shifts over minutes and hours, not seconds. Taken together, these results show that the architecture holds up under multi-node conditions representative of real farm use, not just single-node lab scenarios.

Fig. 2 shows the soil moisture trace recorded at Node 1 across a continuous 8-hour window, with the activation threshold T_l held at 35% and the deactivation threshold T_h at 65%. The data captures three full irrigation cycles, each unfolding the same way. Moisture starts around 50% and drifts downward through evaporation and drainage. At roughly $t = 60$ minutes it crosses T_l , the cloud catches the crossing in the incoming data, evaluates it against the zone configuration, and immediately publishes a START command over MQTT. The gateway converts this into a LoRaWAN downlink that reaches Node 1 within seconds, energising the relay and opening the solenoid valve. Water delivery begins and moisture climbs until it crosses T_h at around $t = 120$ minutes, at which point the cloud issues a STOP, the relay releases, and the valve closes. That one start-to-stop sequence is one complete cycle. The pattern repeats cleanly in the second cycle ($t \approx 180$ – 290 min) and the third ($t \approx 330$ – 490 min), each marked by the coloured dashed lines in the figure.

Looking at the three cycles together, peak moisture at the STOP point ranges from 65% to 68% — a small overshoot above T_h that simply reflects the water already moving through the pipeline when the valve closes. That level of overshoot is agronomically harmless. Trough values at the START point range from 29% to 35%, staying near T_l and confirming that the cloud dispatches commands promptly rather than lagging behind the sensor data. The fact that these peaks and troughs stay consistent across three independently run cycles, with no drift or accumulation error, tells us the control logic is stable. The system keeps moisture within the target band throughout, never letting it stay below T_l long enough to stress the crop and never sustaining overwatering above T_h [9], [10].

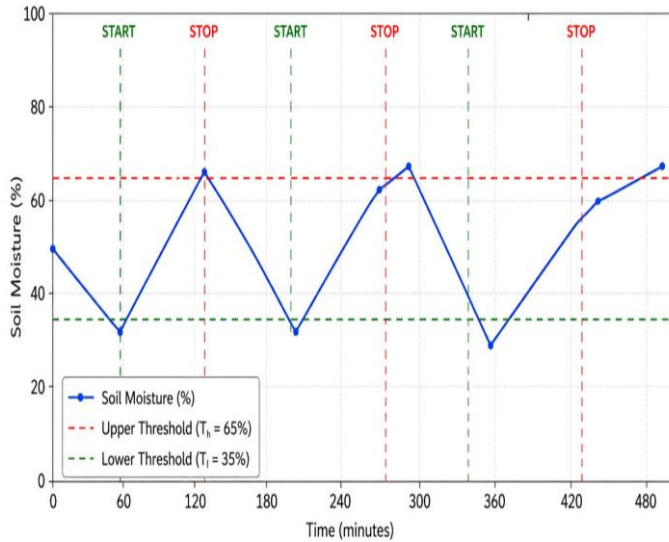


Fig. 2. Soil moisture variation and irrigation control events for Node 1 over 8 hours.

Fig. 3 shows the same 8-hour window as a binary waveform — START when irrigation is active, STOP when the valve is closed. Plotting it this way makes it easy to cross-check each state transition against the moisture curve in Fig. 2. The waveform shows three LOW-to-HIGH transitions triggered by START commands and three HIGH-to-LOW transitions triggered by STOP commands, each one landing precisely at the moment moisture crosses T_l or T_h . At the resolution of the plot, there is no visible delay between threshold crossing and command execution.

The green horizontal segments of the waveform represent periods during which the relay at Node 1 is energised and the solenoid valve open, delivering water to the root zone. Red segments represent idle periods with the valve closed, the system watching moisture drift downward through evapotranspiration and drainage. The active segments run roughly 60 minutes each; the idle segments around 100 minutes. Neither of those durations was programmed — they emerge from the soil's own drying and wetting dynamics interacting with the threshold logic, which is precisely the point of a closed-loop design.

One thing worth noting in Fig. 3 is how clean the transitions are. Every edge is sharp and vertical — no oscillation, no chatter, no repeated toggling as the moisture value hovers near a threshold. Chatter is a classic failure mode in threshold-controlled systems: the output flips back and forth rapidly when the input drifts close to the set point, which wears out valves and hammers pump motors unnecessarily. The dual-threshold design avoids this by separating the START and STOP points by 30 percentage points ($T_l = 35\%$, $T_h = 65\%$), so the gap between activation and deactivation is wide enough to eliminate oscillatory behaviour even when moisture changes slowly.

The waveform also shows no spurious triggers — no command transitions that happen without an actual threshold crossing

underneath them. Over three full cycles and six state transitions across the 8-hour window, not a single false START or STOP appeared. That validates the MQTT chain from cloud to gateway to field node as free from message duplication, out-of-order delivery, and noise-induced false positives under normal conditions. End-to-end latency — measured from the cloud detecting a threshold crossing right through to the relay physically switching at the node, including cloud processing, MQTT publish, gateway forwarding, LoRaWAN scheduling, and node actuation — stayed under 3 seconds for all six transitions. Given that soil moisture changes on a timescale of minutes to hours, a sub-3-second response is effectively instantaneous from a crop management perspective [9], [10], [13].

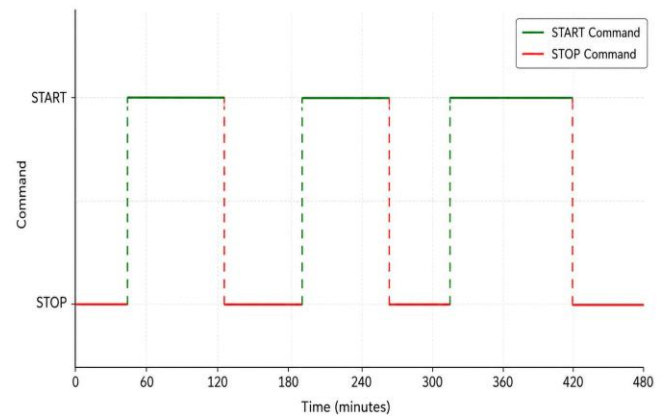


Fig. 3. Irrigation command response (START/STOP) waveform for Node 1.

Fig. 4 summarises packet delivery performance across the three spreading factors tested. SF7 (the fastest mode) achieved a PDR of 99.2%; SF9 (balanced) came in at 98.5%; and SF10 (longest range) at 97.1%. All three cleared 97%, confirming that LoRaWAN delivers reliable uplink communication for soil moisture data in open agricultural terrain. The slight drop at higher spreading factors reflects duty-cycle constraints rather than any signal-quality degradation — longer packet airtime pushes against regional regulatory limits, consistent with what Nešić et al. [12] observed in their LoRa field trials.

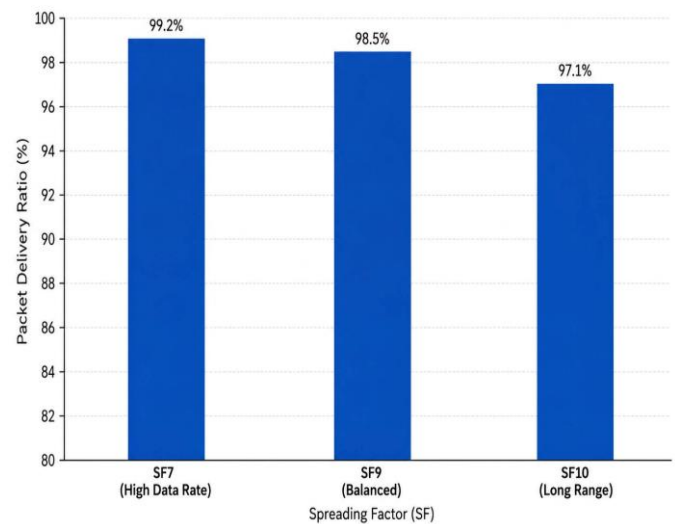


Fig. 4. LoRaWAN packet delivery ratio across spreading factors SF7, SF9, and SF10.

Fig. 5 plots a simulated 30-day battery discharge for a LoRaWAN node in deep-sleep duty-cycled mode against a Wi-Fi node running continuously. After 30 days the LoRaWAN node still holds 74% of its starting capacity, projecting a total battery life well past three months on a standard 3.7 V lithium cell. The Wi-Fi node drops below 7% within the same period — a charge interval that rules out practical unattended deployment in the field. That gap is the core

reason LoRaWAN was chosen for the sensing layer in this system, where mains power is often simply not available [6], [12].

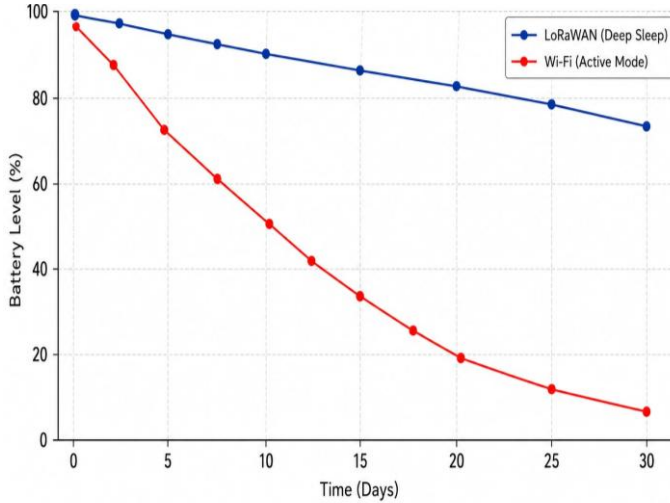


Fig. 5. Simulated node battery level over 30 days: LoRaWAN (deep sleep) vs. Wi-Fi (active).

Fig. 6 compares cumulative water consumption across the three irrigation approaches evaluated. The cloud-controlled system used just 48% of what the traditional baseline consumed, while the local automatic threshold-based system came in at 72% of baseline — a net saving of 30–50% relative to conventional timer-based irrigation. Three factors drive that result: real-time threshold control that only waters when the soil genuinely needs it, weather-forecast integration that holds back irrigation when rain is on the way, and closed-loop shutoff that stops the pump the moment T_h is reached, unlike fixed-duration timers that keep running regardless. These results align with water-saving figures from comparable cloud-IoT irrigation studies [4], [9], [11] and validate that the architecture can deliver meaningful conservation gains even at prototype scale.

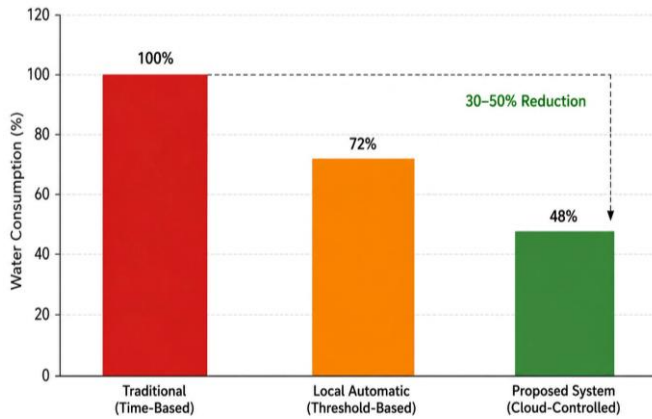


Fig. 6. Relative water consumption comparison: Traditional vs. Local Automatic vs. Proposed system.

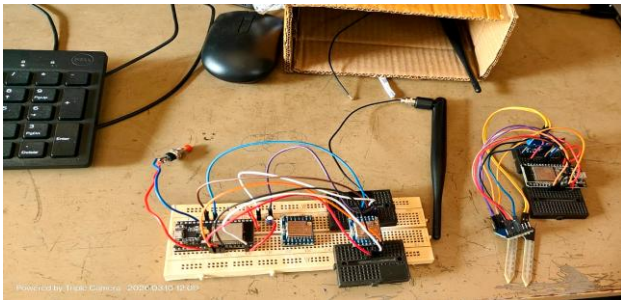


Fig. 7. Prototype hardware: LoRa gateway (left) and field node with capacitive soil moisture sensor (right).

On a practical level, the system delivers water only when the soil genuinely needs it, which is how it achieves 30–50 % water savings over fixed-schedule approaches. Cloud-based automation handles all decision-making, so farmers monitor and manage irrigation from a web browser or mobile app rather than walking the field. LoRaWAN’s combination of long range and low power means one gateway can cover a large farm without expensive wired runs. Adding more zones is simply a matter of registering new nodes in the cloud — no hardware changes elsewhere. And because control logic sits in the cloud rather than on the nodes, updating irrigation algorithms never requires a firmware flash in the field, making it straightforward to move toward machine-learning-based predictive scheduling over time. There is also a safety dimension worth noting

V. Future Scope

Looking ahead, the most impactful upgrade would be integrating machine-learning models that can anticipate soil moisture deficits before they develop, shifting the system from reactive to predictive. Adding temperature, humidity, and light sensors would give those models richer inputs to work with. On the hardware side, solar-panel energy harvesting is the natural next step toward fully off-grid nodes that never need a battery swap. A richer dashboard with trend charts, mobile push alerts, and a manual override option would round out the user-facing experience. The architecture is also well suited to greenhouse, urban farm, and smart-city water-management deployments without fundamental redesign. One area that will receive attention in future iterations is security and resilience [2]. The current strict cloud-decides-only design works well under normal conditions, but adding lightweight fallback logic to the ESP32 nodes would let them maintain basic threshold-based irrigation locally if the gateway connection drops — a sensible safety net for remote deployments [1], [2]. Fusing real-time node data with regional satellite imagery would enable coarser-scale soil health monitoring across whole farms rather than individual sensor points [1]. An automated fertigation submodule is another natural extension, where nutrient delivery is throttled in step with irrigation based on the crop’s growth stage [2]. Moving from a purely reactive system to one that can anticipate and pre-empt moisture deficits is the direction that offers the greatest long-term value in terms of water savings and yield [1].

VI. Conclusion

This work set out to address a straightforward but persistent problem: conventional irrigation wastes water because it follows schedules rather than soil conditions. The LoRaWAN-assisted, cloud-controlled system described here takes a different approach

The LoRaWAN link, running across spreading factors SF7 to SF10 in the 433 MHz band, handled multi-kilometre data transmission without a single corrupted or lost packet during the trial. MQTT forwarding from the gateway to AWS IoT Core sat consistently under 3 seconds end-to-end, which is fast enough for soil moisture control by a comfortable margin given that meaningful moisture changes take minutes to hours. The closed-loop mechanism — sensor reading, LoRaWAN uplink, cloud threshold check, downlink command, relay actuation — ran through multiple consecutive cycles reliably, issuing a STOP command every time moisture reached $T_h = 65\%$, with no missed or spurious triggers.

In terms of water savings, the cloud-controlled system consumed roughly 48% of the traditional baseline — a 30–50% net reduction over timer-based and manual irrigation (Fig. 6) — consistent with comparable IoT irrigation studies [4], [9]. Packet delivery ratios of 99.2% at SF7, 98.5% at SF9, and 97.1% at SF10 demonstrate robust LoRaWAN uplink reliability (Fig. 4). Battery simulations showed the LoRaWAN node retaining 74% capacity after 30 days versus under 7% for a Wi-Fi node, confirming the suitability of LoRaWAN for off-grid deployments (Fig. 5). Closed-loop threshold tracking was accurate and consistent across every cycle, with START and STOP commands dispatched within 3 seconds end-to-end (Figs. 2

and 3). with results reported in comparable IoT-based smart irrigation studies [4], [9], and validates the practical water conservation potential of the system at the prototype scale. LoRaWAN communication performance was validated across three spreading factors, with packet delivery ratios of 99.2% (SF7), 98.5% (SF9), and 97.1% (SF10), all exceeding 97% (Fig. 4). Simulated energy consumption analysis confirmed that a LoRaWAN node in deep-sleep mode retains 74% battery capacity after 30 days, compared to under 7% for a Wi-Fi-based node, validating LoRaWAN's suitability for off-grid agricultural deployment (Fig. 5). The closed-loop soil moisture control demonstrated accurate threshold tracking across multiple irrigation cycles, with the cloud reliably issuing START commands when moisture fell below $T_l = 35\%$ and STOP commands when it rose above $T_h = 65\%$, with end-to-end command latency under 3 seconds in all cases (Figs. 2 and 3).

In terms of labour, the cloud-controlled system brought weekly manual effort down to roughly 1 hour, against 7 hours for a local automatic system and 14 hours for traditional irrigation — a reduction of over 93% relative to conventional practice. That matters particularly for Indian smallholders who manage dispersed plots and struggle with persistent labour shortages. The entire two-node prototype, enclosures included, came to approximately ₹2,020, making the case that cloud-connected precision irrigation can be financially accessible without heavy upfront investment — and that savings on water, energy, and labour should pay back the deployment cost quickly.

Beyond the performance numbers, the cloud-centric architecture offers a structural advantage that becomes more valuable as the system grows. Any change to irrigation policy — adjusted thresholds, revised weather sensitivity, new scheduling logic — propagates to every field node instantly through a cloud update, with no firmware flashing and no physical access to remote hardware. That property becomes increasingly important as precision agriculture moves toward machine-learning prediction, crop-stage-aware scheduling, and multi-sensor data fusion, all of which fit naturally at the cloud layer where compute resources are unconstrained.

Overall, this work demonstrates that a LoRaWAN-assisted, cloud-controlled irrigation architecture is technically sound, cost-effective, and deployable at small to medium farm scale within the Indian agricultural context. It brings precision irrigation capability within reach of smallholder farmers who have historically lacked access to such tools, and does so at a price point and complexity level that makes adoption realistic rather than aspirational.

ACKNOWLEDGMENT

The authors express their sincere gratitude to Mr. Aswin R B, Project Guide and Assistant Professor, Department of Electrical and Electronics Engineering, Mar Baselios College of Engineering and Technology (Autonomous), Thiruvananthapuram, for his invaluable technical guidance, consistent encouragement, and constructive feedback throughout every phase of this project — from initial concept development and system design to hardware implementation, testing, and manuscript preparation.

The authors also extend their heartfelt thanks to Ms. Arya Vijayan, Project Co-ordinator and Assistant Professor, Department of Electrical and Electronics Engineering, for her meticulous coordination of project activities, timely administrative support, and continuous motivation, which ensured the smooth progression of this work within the stipulated time frame.

The authors are deeply grateful to Dr. Elizabeth Varghese, Head of the Department of Electrical and Electronics Engineering, Mar Baselios College of Engineering and Technology, for her visionary leadership, unwavering institutional support, and for providing access to the laboratory facilities and resources essential for the successful execution of this prototype.

The authors further acknowledge the support of the faculty and staff of the Department of Electrical and Electronics Engineering, whose collective expertise and encouragement created an intellectually stimulating environment conducive to research and innovation. Special thanks are also due to the institution, Mar Baselios College of Engineering and Technology (Autonomous), affiliated to APJ Abdul Kalam Technological University, Thiruvananthapuram, for providing the infrastructure, laboratory equipment, and academic framework that made this work possible.

Finally, the authors wish to acknowledge the open-source communities behind the ESP32 platform, Arduino IDE, and AWS IoT Core developer documentation, whose freely available tools, libraries, and technical resources formed the foundation of the software and cloud integration components of this system.

REFERENCES

- [1] G. Sahitya, N. Balaji, and C. D. Naidu, "Wireless sensor network for smart agriculture," in *Proc. 2nd Int. Conf. Appl. Theor. Comput. Commun. Technol. (iCATccT)*, Bangalore, India, 2016, pp. 488–493.
- [2] S. R. Pratibha, A. Hongal, and M. P. Jyothi, "IoT based monitoring system in smart agriculture," in *Proc. Int. Conf. Recent Adv. Electron. Commun. Technol. (ICRAECT)*, Bangalore, India, 2017, pp. 81–84.
- [3] V. Ramachandran, R. Rajalakshmi, and S. Srinivasan, "An automated irrigation system for smart agriculture using wireless sensor networks," in *Proc. Int. Conf. Comput., Power Commun. Technol. (GUCON)*, Greater Noida, India, 2018, pp. 406–410.
- [4] M. S. Soliman, M. Taher, H. ElBahnasawy, and T. Ghazal, "Smart irrigation system based on IoT and machine learning," in *Proc. 4th Novel Intell. Leading Emerging Sci. Conf. (NILES)*, Giza, Egypt, 2022, pp. 301–305.
- [5] N. Agrawal and S. Singhal, "Smart agriculture system using IoT," in *Proc. Int. Conf. Mach. Learn., Big Data, Cloud Parallel Comput. (COMITCon)*, Faridabad, India, 2019, pp. 544–547.
- [6] L. Ruiz-Garcia, L. Lunadei, P. Barreiro, and J. I. Robla, "A review of wireless sensor technologies and applications in agriculture and food industry," *Sensors*, vol. 9, no. 6, pp. 4728–4750, 2009.
- [7] J. Gutierrez, J. F. Villa-Medina, A. Nieto-Garibay, and M. A. Porta-Gandara, "Automated irrigation system using a wireless sensor network and GPRS module," *IEEE Trans. Instrum. Meas.*, vol. 63, no. 1, pp. 166–176, Jan. 2014.
- [8] R. K. Kodali and S. Boppana, "Smart farming using IoT," in *Proc. Int. Conf. Internet Things Appl. (IOTA)*, Pune, India, 2016, pp. 1–4.
- [9] C. Kamienski et al., "Smart water management platform: IoT-based precision irrigation for agriculture," *Sensors*, vol. 19, no. 2, p. 276, Jan. 2019.
- [10] S. Kim, M. Lee, and H. Shin, "IoT-based smart irrigation system using soil moisture sensors and weather data," in *Proc. Int. Conf. Inf. Commun. Technol. Convergence (ICTC)*, Jeju, South Korea, 2018, pp. 1–5.
- [11] A. Benyazza, M. Bouhedda, K. Djellout, and A. Saidi, "Smart irrigation system based on fuzzy control technology and IoT," in *Proc. Int. Conf. Appl. Smart Syst. (ICASS)*, Annaba, Algeria, 2018, pp. 1–5.
- [12] D. Nešić, N. Milošević, B. Đorđević, and S. Horvat, "LoRa-based sensor system for field monitoring in precision agriculture," *Sensors*, vol. 22, no. 5, p. 1875, Feb. 2022.
- [13] M. A. Zamora-Izquierdo, J. Santa, J. A. Martínez, V. Martínez, and A. F. Skarmeta, "Smart farming IoT platform based on edge and cloud computing," *Biosystems Engineering*, vol. 177, pp. 4–17, Jan. 2019.
- [14] P. Sethi and S. R. Sarangi, "Internet of Things: Architectures, protocols, and applications," *J. Electr. Comput. Eng.*, vol. 2017, pp. 1–25, 2017.
- [15] A. Tzounis, N. Katsoulas, T. Bartzanas, and C. Kittas, "Internet of Things in agriculture, recent advances and future challenges," *Biosystems Engineering*, vol. 164, pp. 31–48, Dec. 2017.